

Data Science

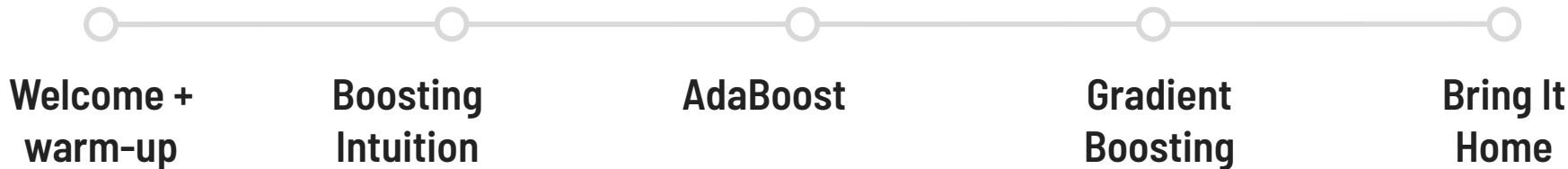
Ensemble Methods: Boosting

Advanced Modeling



GENERAL ASSEMBLY

Lesson roadmap



Learning objectives



1

Understand the differences between bagging and boosting.

2

Learn the pros and cons to using boosting models.

3

Learn the effect of boosting on the bias-variance trade-off.

4

Learn the math and procedure for AdaBoost, a classic boosting model.

5

Understand the differences between AdaBoost and gradient-boosting models.



Warm Up

Fix the last mistake

How would you tutor your friend?

Imagine you're tutoring a friend who keeps getting math problems wrong. You notice a pattern: they mess up fractions, but not multiplication.

How would you help them improve?

Boosting

Focus on fixing our mistakes!

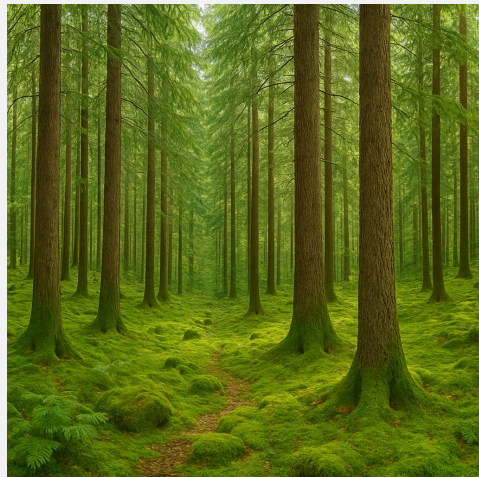
Recap: there are 3 main ensemble learning methods

1. Bagging
2. Boosting - **this session**
3. Stacking



Bagging and Random Forests

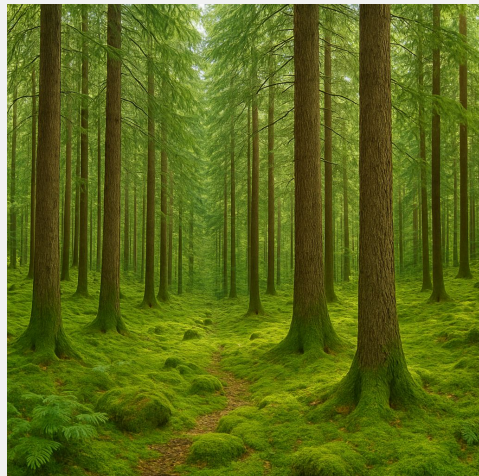
We ensembled overfit decision trees in order to reduce variance. That is, **averaging overfit models together was a way to reduce the overfittedness of the overall model.**



Bagging and Random Forests

An added bonus is that these ensemble models are **parallelizable**: the individual base models (learners) can be trained independently and simultaneously across multiple processors or machines.

Why is that a benefit?



Boosting

In contrast to bagging, **boosting** is another ensemble technique that solves the opposite problem:

We're going to take **weak learners** and **successively boost them to reduce bias.**

Typically, these weak learners are going to be very high-bias decision trees. Specifically, they're most often 1-split **decision stumps**. (i.e., **max_depth=1**)



Random Forest

overfit trees

Bagging

underfit "stumps"

AdaBoost

Boosting

underfit
(high bias)

just right
(good tradeoff)

overfit
(high variance)

Ok, but why trees?

Graphic taken from our Rosarium philosophorum, *The Elements of Statistical Learning*, by Hastie, Tibshirani, and Friedman (praise be unto them)

TABLE 10.1. Some characteristics of different learning methods. Key: ▲ = good, ◆ = fair, and ▼ = poor.

Characteristic	Neural Nets	SVM	Trees	MARS	k-NN, Kernels
Natural handling of data of “mixed” type	▼	▼	▲	▲	▼
Handling of missing values	▼	▼	▲	▲	▲
Robustness to outliers in input space	▼	▼	▲	▼	▲
Insensitive to monotone transformations of inputs	▼	▼	▲	▼	▼
Computational scalability (large N)	▼	▼	▲	▲	▼
Ability to deal with irrelevant inputs	▼	▼	▲	▲	▼
Ability to extract linear combinations of features	▲	▲	▼	▼	◆
Interpretability	▼	▼	◆	▲	▼
Predictive power	▲	▲	▼	◆	▲

AdaBoost

Math inbound

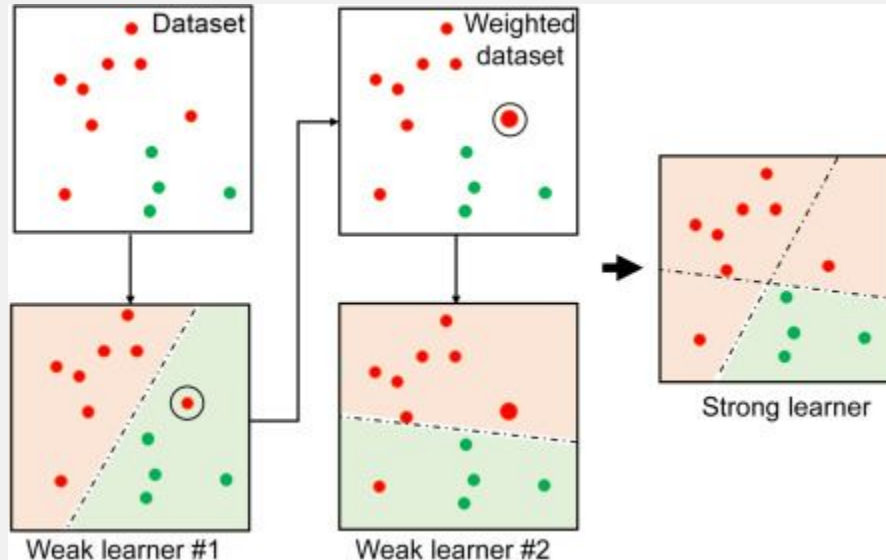
AdaBoost

The “simplest” (lol) form of boosting is known as the **AdaBoost**.

The general idea of boosting is to fit successive models that each improve upon the one before them.

After each model step in the AdaBoost, the **next model pays more attention to the parts of the data it got the most incorrect**. By focusing special attention on these parts, it spends most of its time learning the hardest parts of your data rather than the easiest, most separable parts.

Visually, this is what AdaBoost looks like



AdaBoost Algorithm (as math-free as possible)

1. Initialize the observation weights $\mathbf{w}$ to each be $1/n$ (i.e., all weighted the same)
2. For $m = 1, \dots, M$:
 - a. Fit a **weighted** underfit model G_m
 - b. Compute the **weighted** misclassification rate
 - c. Compute the model weight α_m that determines this model's overall importance to the ensemble
 - d. Reweight the observations as weight vector $\mathbf{w}$. This is a vector in which the “easy” observations have a low weight and the “hard” observations get a large weight
3. The final model G is the weighted amalgamation of all the G_m s and the α_m s.

AdaBoost (mathfull notation)

When talking about boosting (and also for SVM models), when we talk about binary classification, we let $y \in \{-1, 1\}$ instead of the usual $y \in \{0, 1\}$ so we can take advantage of a few math tricks.

In what follows...

- $G_m(x)$ is a model, and so it can take only values -1 and 1
- $I(.)$ is the indicator function. ie, $I(A)$ is 1 if A is true and 0 otherwise.
- $\text{sign}(.)$ is the “signum” function. ie, $\text{sign}(x) = 1$ if x is positive and -1 if it's negative.

AdaBoost Algorithm (for all you math nerds out there)

Algorithm 10.1 *AdaBoost.M1*.

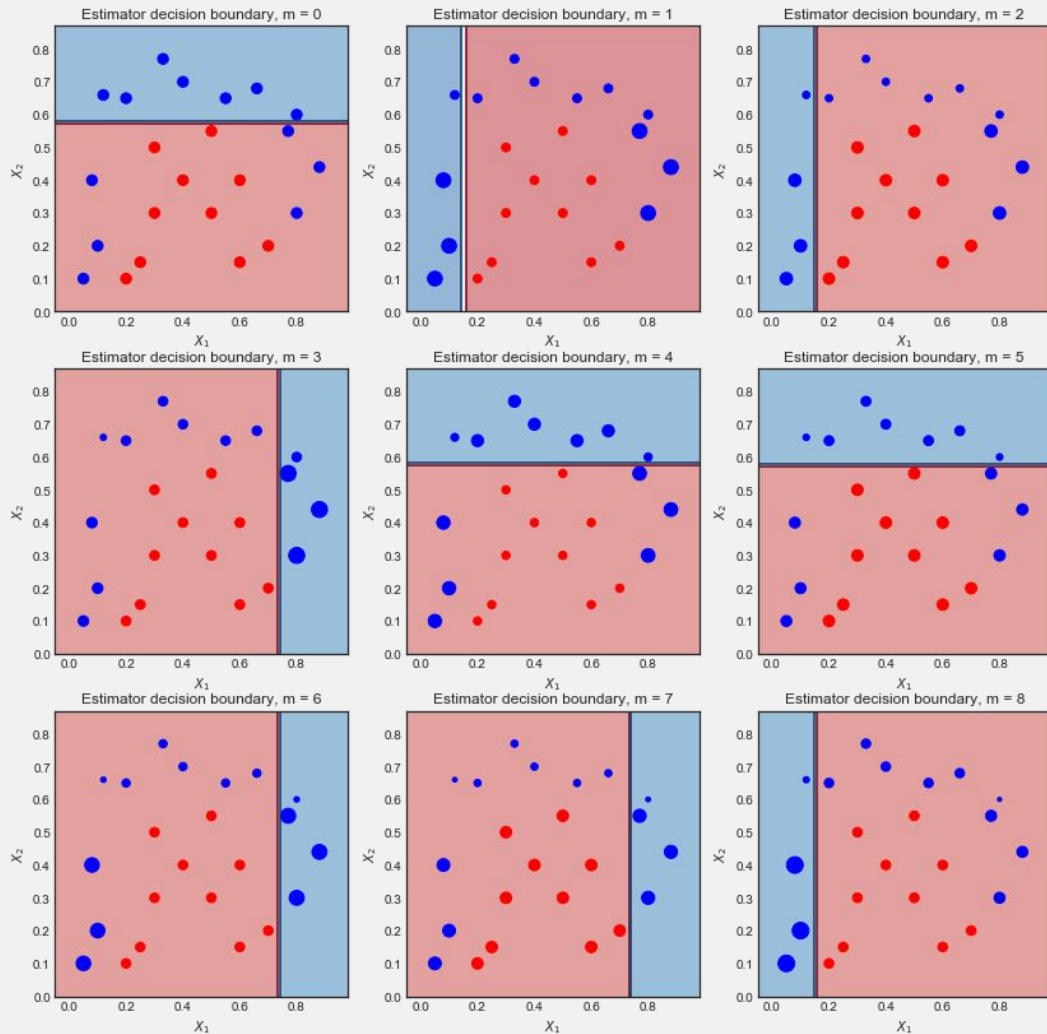
1. Initialize the observation weights $w_i = 1/N$, $i = 1, 2, \dots, N$.
2. For $m = 1$ to M :
 - (a) Fit a classifier $G_m(x)$ to the training data using weights w_i .
 - (b) Compute

$$\text{err}_m = \frac{\sum_{i=1}^N w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^N w_i}.$$

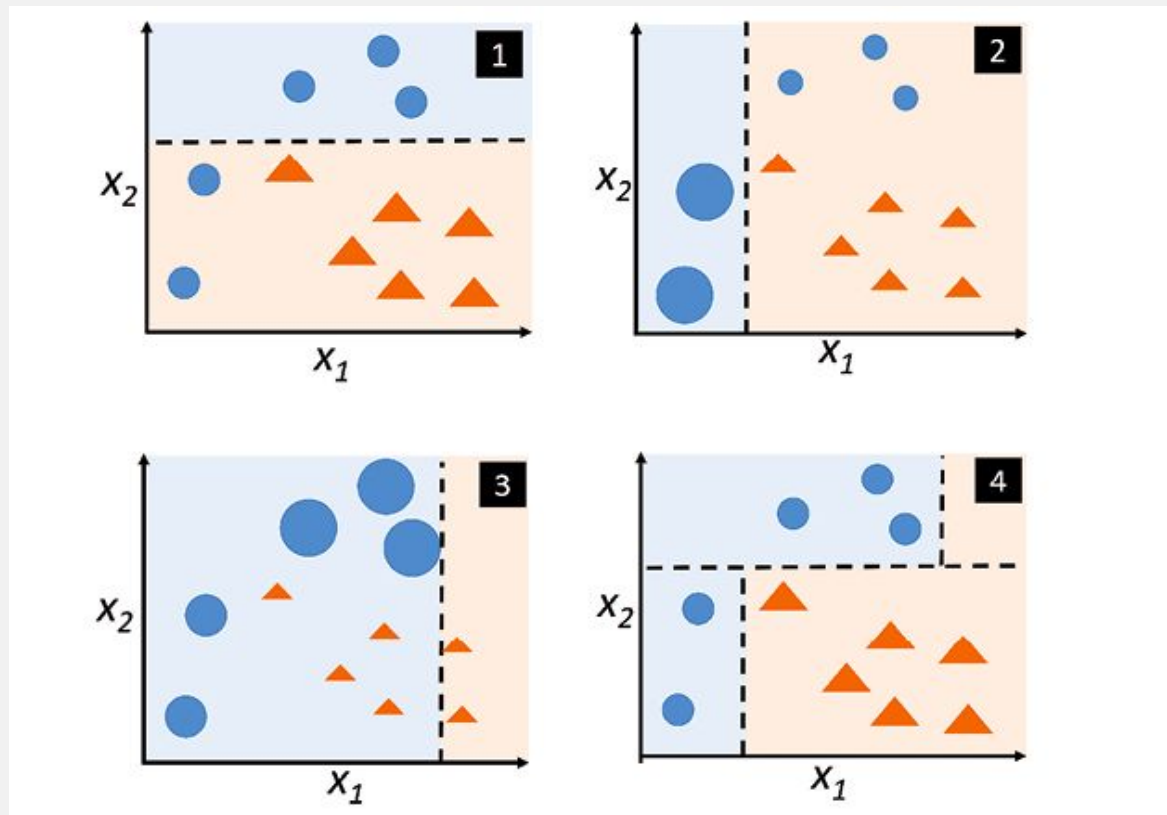
- (c) Compute $\alpha_m = \log((1 - \text{err}_m)/\text{err}_m)$.
 - (d) Set $w_i \leftarrow w_i \cdot \exp[\alpha_m \cdot I(y_i \neq G_m(x_i))]$, $i = 1, 2, \dots, N$.
3. Output $G(x) = \text{sign} \left[\sum_{m=1}^M \alpha_m G_m(x) \right]$.

(As always,
clipped directly
from ESL)

AdaBoost Viz



Another Viz



AdaBoost

Let's see it for ourselves!

- Open **02-ensembles-boosting.ipynb**



Gradient Boost

(oh God there's more math and it's worse somehow)

AdaBoost vs Gradient Boost

Both are similar boosting techniques, but gradient boost takes a slightly different approach.

Where **AdaBoost** improves successive models by focusing more on the hard-to-reach parts of our data...

Gradient Boost focuses on the observations with the worst residuals and successively corrects the model to “fix” them.

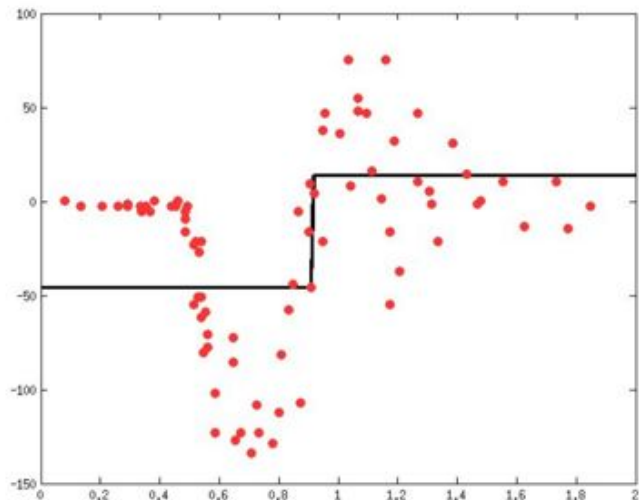


The Gradient Tree Boost Algorithm (math lovers keep OUT)

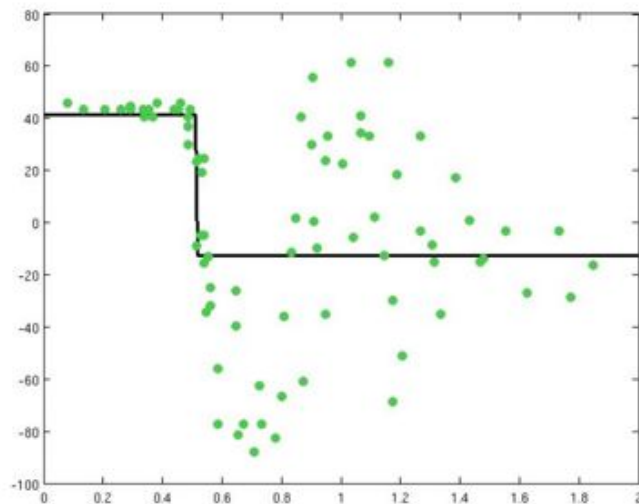
1. Initialize the base model f_θ to be the null model.
2. For $m = 1, \dots, M$:
 - a. Compute the ~~gradient of the loss function~~ “residuals”, $\mathbf{r}$.
 - i. There's a special way of computing residuals for classification problems called **deviance**, and let us never speak of it again!
 - b. Fit an underfit tree model, except with $\mathbf{r}$ instead of $\mathbf{y}$ as the response variable.
 - c. For each terminal region (i.e., partition of feature space) computed in 2b, compute a new residual γ that gets you the best loss function with respect to $f_{m-1} + \gamma$ (i.e., this is readjusting the predictions based on how bad they were)
 - d. Update $f_m = f_{m-1} + \text{adjustments involving } \gamma$
3. The final model is just f_M

A quick little visual aid of GradBoost

Learn a simple predictor...



Then try to correct its errors



Gradient Tree Boosting algorithm, behold!

Algorithm 10.3 *Gradient Tree Boosting Algorithm.*

1. Initialize $f_0(x) = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, \gamma)$.

2. For $m = 1$ to M :

(a) For $i = 1, 2, \dots, N$ compute

$$r_{im} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=f_{m-1}}.$$

(b) Fit a regression tree to the targets r_{im} giving terminal regions R_{jm} , $j = 1, 2, \dots, J_m$.

(c) For $j = 1, 2, \dots, J_m$ compute

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma).$$

(d) Update $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$.

3. Output $\hat{f}(x) = f_M(x)$.

Lifted from ESL



Gradient Tree Boosting algorithm, behavior

Algorithm 10.3 Gradient Tree Boosting Algorithm

1. Initialize $f_0(x) = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, \gamma)$.
2. For $m = 1$ to M :
 - (a) For $i = 1, 2, \dots, N$ compute $r_{im} = -\frac{1}{L''(f_{m-1}(x_i))} L'(y_i, f_{m-1}(x_i))$.
 - (b) Fit a regression tree to r_{im} giving terminal regions R_{jm} .
 - (c) Find γ_{jm} that minimize $\sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma)$.
3. Output $f_M(x) = f_0(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$.

Lifted from ESL



A note on these algorithms...

We're getting to the point where the algorithms themselves are far beyond the mathematical scope of our class, and are really most only the interest of sad, lonely academics.

For us, the key things to remember are the gists of why these algorithms do what they do, and the basics of how.

Failing that, we can just ham-fist them in Python, so...

Gradient Boosting

Let's see it for ourselves!

- Open **02-ensembles-boosting.ipynb**



XGBoost

Another one???

eXtreme Gradient Boosting

Maybe the most popular form of boosting today is the **Extreme Gradient Boosting** algorithm, aka **XGBoost**.

It is algorithmically similar to **gradient boost** but has been shown to generally get better results.



XGBoost

More importantly, however, is that the algorithm is engineered to be **parallelizable**, and its implementation in Python can also be **distributed**!

This makes it a very popular model for Big Data problems.

- XGBoost Tutorials

- Introduction to Boosted Trees
- Introduction to Model IO
- Learning to Rank
- DART booster
- Monotonic Constraints
- Feature Interaction Constraints
- Survival Analysis with Accelerated Failure Time
- Categorical Data
- Multiple Outputs
- Random Forests(TM) in XGBoost
- Distributed XGBoost on Kubernetes
- Distributed XGBoost with XGBoost4J-Spark
- Distributed XGBoost with XGBoost4J-Spark-GPU
- Distributed XGBoost with Dask
- Distributed XGBoost with PySpark
- Distributed XGBoost with Ray
- Using XGBoost External Memory Version
- C API Tutorial



XGBoost: The Pros & Cons

Pros:

- ☐ Really good, Kaggle-winning predictions
- ☐ Optimized, regularized gradient boosting
- ☐ Handles missing data natively!
- ☐ Scales to massive data

Cons:

- ☐ More prone to overfitting, even among tree-based models
- ☐ Can be very slow to fit on large data

Implementing XGBoost

XGBoost is not implemented in **scikit-learn**. Instead, it has its own library. So in order to use it, we'll need to:

```
pip install xgboost
```



Boosting - XGBoost

Let's see it for ourselves!

- Open **02-ensembles-boosting.ipynb**



Bring It Home

Bagging vs Boosting

Ensemble Method	Learner Focus	Goal	Typical Learner
Bagging	Independent	Reduce Variance	Deep Trees
Boosting	Sequential	Reduce Bias	Decision Stumps

Why not always use this?

ADVANTAGES OF CV

- You get a measure of bias/variance that's more robust to outliers
- You don't "throw away" data - every observation gets to be used as part of model training

DISADVANTAGES OF CV

- Since the same data are used to compute the MSE in each fold, by averaging them you're actually introducing some bias into your estimate. (People typically ignore this though).
- You need to carry out model fitting k times! This is a **huge** disadvantage!!

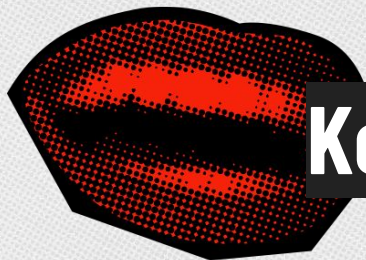
Boosting

Pros:

- ❑ Top-tier performance
- ❑ You get all the same benefits associated with tree-based models

Cons:

- ❑ All hope of interpretation, gone!
- ❑ Computational complexity is the highest we've ever seen - can cause models to fit slowly!
- ❑ Because of serial nature of algorithm, parallelization is impossible



Key takeaways



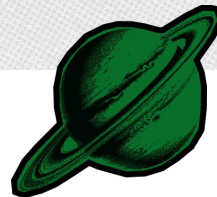
Boosting reduces bias by correcting mistakes

The general idea is to fit successive models that each improve upon the one before them.



Boosting is sequential

Each model pays more attention to the parts of the data it got the most incorrect



There are many Boosting models

AdaBoost, Gradient Boost, and XGBoost are all variants

